Context Is the New Weight: Trading the Attention Sink for a Constant Working Memory via Windowed Finetuning

Anonymous submission

Abstract

Causal language models develop an *attention sink*: heads route a large fraction of their attention onto the first one or two tokens, regardless of content. We argue this sink is not an accident but a direct consequence of the triangular causal mask, early positions are visible to every later query, so they accumulate attention, and that it is precisely what makes a pretrained model fragile to a bounded context: naively restricting attention to a sliding window deletes position 0 and the model collapses. We instead finetune the attention itself under a sliding-window mask, so the model operates with a *constant working memory* of the W most-recent tokens and must offload everything older into its weights, *context becomes weight*. We study three variants: a plain **windowed** model, a **startup** model with a trainable cold-start prompt, and a **persistent-prompt** model whose prompts are always attendable (trainable attention sinks). On Qwen2.5-0.5B, windowed finetuning recovers near-full-context language-modelling quality at a 256-token window and drives the position-1 sink from 57% of heads to 0%; the sink relocates onto the startup prompt. On the hybrid Qwen3.5-9B (only $1/4$ of layers are softmax attention), finetuning just those 8 layers under a window matches the model's full-context perplexity at a 256-token window and *beats StreamingLLM without keeping any sink tokens*. A train-window $\times$ inference-window study yields a simple deployment rule, *train small, deploy anywhere*, and the persistent-prompt variant turns the sink into 64 trainable always-on registers that absorb a controllable 19% of attention.

1 Introduction

Transformer language models are trained with a causal (triangular) attention mask: a query at position t may attend only to keys at positions $\leq t$. A robust empirical phenomenon emerges under this mask, the *attention sink* (Xiao et al. 2024): across heads and layers, a disproportionate share of attention mass lands on the very first token(s) of the sequence, even when those tokens carry no relevant information. The sink is now understood as a "no-op" that heads use to dump attention they do not need, and exploiting it (keeping a few initial tokens) is what lets streaming inference work at all (Xiao et al. 2024; Han et al. 2024).

We take the sink seriously as a *cause*, not just a curiosity. Two observations motivate this paper. First, the sink is an artifact of the mask geometry: under a triangular mask the first token is in the receptive field of *every* later query, so it is structurally positioned to accumulate attention; the data, by contrast, dilute the first token's importance as context grows. Second, the sink is exactly what breaks a model when we try to bound its memory. If we want a *constant working memory*, attention restricted to the W most-recent tokens, so that compute and KV-cache are $O(W)$ rather than $O(t)$, then a plain sliding window deletes position 0, removes the sink, and the pretrained model collapses.

The standard response is to keep the sink at inference time (StreamingLLM: retain a few initial "sink" tokens plus the recent window) (Xiao et al. 2024). We ask a different question: *can we finetune the sink away?* If the model is trained under the sliding-window mask, every token must learn to predict from a bounded window, and any long-range information it still needs has to be written into the *weights* rather than read from far-away keys. This is the sense in which "context is the new weight": adaptation that an in-context model performs by attending to distant tokens is, after windowed finetuning, performed by the parameters.

We make three contributions. First, we **disentangle two types of attention sink by structure and cause**: a concentrated sink on the first token and a sparse sink on low-information separators (commas, periods), orthogonal to the functional split (no-op vs. broadcast) of Fesser et al. (2026). We trace them to two different mechanisms, the concentrated sink to the triangular mask's exposure imbalance and the no-op sink to content-driven bounded-memory token usage (Sec. 2–3), and show by finetuning that the positional sink is learned and removable (a windowed mask drives it to zero) while the no-op sink remains and is leaned on more. Second, we introduce three windowed-finetuning variants: **windowed**, **startup** (a trainable cold-start prompt), and **persistent-prompt** (always-on trainable registers), giving a model a constant working memory (Sec. 3). Third, on Qwen2.5-0.5B and the hybrid Qwen3.5-9B we show that windowed finetuning recovers near-full-context perplexity at a 256-token window, beats StreamingLLM without sink tokens, and obeys a clean "train small, deploy anywhere" rule (Sec. 3).

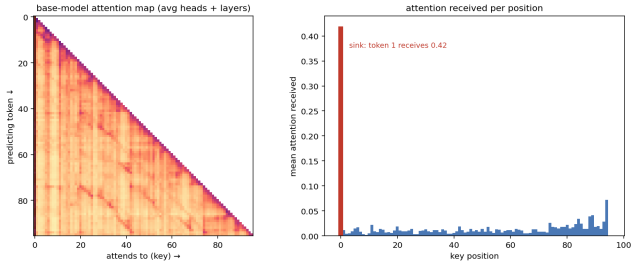

Figure 1: Type 1 (positional) sink in a base causal LM. *Left:* attention map (averaged over heads and layers); the first column is a bright vertical stripe. *Right:* mean attention received per key position, token 1 absorbs 0.42, every other token < 0.05.

2 Categorizing Attention Sinks

	function (Fesser et al. 2026)	
structure (ours)	no-op	broadcast
Type-1 (concentrated)	null-route anchor	global register
Type-2 (sparse)	delimiter scratch	span summary

Table 1: Two *orthogonal* categorizations of attention sinks. The *functional* axis of Fesser et al. (2026) asks what a sink computes, a *no-op* that suppresses its update by routing to a null token (negligible value norm) vs. a *broadcast* that aggregates and redistributes global information (low-rank output). Our *structural* axis asks the sink's spatial pattern, a *concentrated* sink on the first token (Type-1) vs. a *sparse* sink spread over low-information separators (Type-2). The axes cross-cut (each cell is realizable): a sink of either structure can serve either function, so the functional split alone cannot assign a *cause*. The structural split can, Type-1 is a mask artifact (H2), Type-2 is content-driven, which is what lets us remove one while keeping the other. We measure this functional character empirically across training schemes in Table 3.

Attention sinks admit more than one categorization. We propose a *structural* one, by the spatial pattern of the sink itself, distinguishing a *concentrated* sink from a *sparse* sink. They differ in *where* the attention lands and in *what* drives it, and the rest of the paper turns on telling them apart.

Type 1, the concentrated (first-token) sink. A large, almost content-independent share of every query's attention *concentrates* on the first one or two tokens of the sequence, regardless of their content. Figure 1 shows this for a base model: the leftmost column of the attention map is a bright vertical stripe, and token 1 absorbs on average 0.42 of the attention mass, an order of magnitude more than any later key, across a majority of heads. This is the sink exploited by StreamingLLM (Xiao et al. 2024): it is tied to absolute *position*, not to meaning.

Type 2, the sparse (separator) sink. A second sink is *sparse* rather than concentrated: it spreads across semantically near-empty separator tokens *within* the sequence, punctuation such as commas and periods, delimiters, and other filler. These tokens carry little predictive information of their own; a head with nothing to retrieve at a given step dumps its attention onto the nearest such token, using it as a local "no-op" or scratch register. Unlike Type 1, the no-op sink is distributed throughout the text and keyed to token *identity* rather than to absolute position.

Functional vs. structural categorization. Prior work separates sinks by *function*, but not by *structure* or by *cause*. Fesser et al. (2026) distinguish two algorithms behind a sink, an adaptive *no-op* that suppresses the update by routing to a null token (negligible value norm) and a *broadcast* that aggregates and redistributes global information (low-rank output), while earlier accounts simply retain the initial tokens at inference (Xiao et al. 2024; Han et al. 2024). We categorize *orthogonally and structurally*, by the sink's spatial pattern: a *concentrated* first-token sink (Type 1) vs. a *sparse* separator sink (Type 2). The two axes cross-cut (Table 1), a sink of either structure can serve either function, so the functional split alone cannot assign a *cause*. The structural split can, and this is our central claim (H1): the two structural types arise from *two different mechanisms*. Type 1 is an artifact of the triangular mask's positional exposure imbalance (H2): a property of the *architecture* that disappears when the mask changes. Type 2 is content-driven, keyed to token *identity*: a property of the *data and the model's use of it* that persists, indeed is leaned on *more*, once Type 1 is removed (H3). Separating the sink *by position* is what lets us remove the harmful, mask-induced one while keeping (and exploiting) the useful one. We state them as testable hypotheses (§3), each presented with its mechanism and evidence together.

3 Hypotheses and Evidence

We state four hypotheses, each a claim presented with its mechanism and the experiment that tests it; we introduce the finetuning variant each hypothesis needs where it is used. We evaluate on Qwen2.5-0.5B (a pure-softmax proof of concept) and the hybrid Qwen3.5-9B; finetuning is continued pretraining on WikiText-103 (Merity et al. 2017) (language-modelling loss only) with headline window $W{=}256$.

3.1 H1: The Sink Is Two Disentanglable Types

Hypothesis. We hypothesize that what is usually called "the attention sink" is in fact two phenomena that can be separated. *Type 1* is *concentrated*, a large, content-independent share of attention parked on position 0. *Type 2* is *sparse*, attention spread across semantically near-empty separator tokens (commas, periods, delimiters) used as local no-op registers, keyed to token *identity* rather than to position. Prior work separates sinks by *function* (Fesser et al. 2026) but not by structure or cause; we claim the two *structural* types arise from *two different mechanisms* (H2 and the data, respectively) and can therefore be removed or kept independently.

Method. To test this we finetune under a sliding-window mask $M_{\text{win}}(t,k) = \mathbb{1}[\,t - W < k \le t\,]$ that replaces the causal mask $M_{\text{causal}}(t,k) = \mathbb{1}[k \le t]$ (Fig. 2), so each token

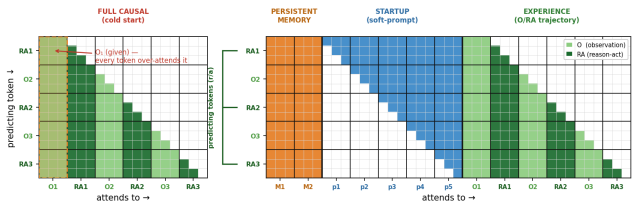

Figure 2: Windowed attention finetuning. The triangular mask (left) is replaced by a sliding window (a band of width W), giving each token a constant working memory of the recent W tokens; older context must be carried by the weights. **Startup** and **persistent-prompt** prepend trainable prompts that fill the cold start or stay always-attendable, respectively.

attends only to the W most-recent positions. We compare the **base** model, a full-causal continued-pretraining control (**triangle**), the plain **windowed** model, and a **startup** variant that prepends P trainable soft prompts filling the window's cold start, a learned replacement for the sink token that slides out of the window for deep tokens.

The two sink types are *separable*. On Qwen2.5-0.5B we measure, under each scheme's deployment, the attention received by the first token (the concentrated Type-1) versus low-information separators (the sparse Type-2), and the broadcasting entropy of the per-position attention (Table 2, Fig. 3). They move *independently*: continued full-causal (triangle) training keeps, even slightly intensifies, Type-1 ($0.37 \rightarrow 0.38$), while the **startup** variant drives Type-1 to 0.004 and the Type-2 mass *rises* to 0.40. One type can be removed while the other is kept, which is what it means for them to be disentanglable. The split is robust to the Type-2 definition: the same shift appears whether Type-2 is a fixed punctuation list or the bottom-20% unigram-surprisal tokens, and the attention–surprisal correlation sharpens from -0.07 to -0.16, it is not an artifact of the punctuation proxy.

Structural meets functional. The two structural types also differ *functionally*, instantiating the cross-tab of Table 1 (Fesser et al. 2026) with measurements: across all training schemes the concentrated Type-1 sink carries a low relative value norm ($\sim$0.4, the no-op signature, it is read precisely to discard the update), while the sparse Type-2 separators carry near-normal value ($\sim$0.8–0.9) and broadcast content (Table 3). As windowed and startup training drain Type-1, attention shifts onto the higher-value Type-2 tokens, structurally concentrated$\rightarrow$sparse and functionally no-op$\rightarrow$broadcast in step. Two caveats keep this honest: pos-0 is not a *pure* no-op (heavily attended, it still injects $\sim 15\%$ of the base model's output energy), and our *uniform-window* scheme reduces Type-1 *without* the Type-2 spread, it sinks on each window's relative boundary instead.

3.2 H2: The Concentrated Sink Is a Mask Artifact

Hypothesis. Under a length-C lower-triangular mask a query early in the sequence has almost no tokens to attend, while position 0 sits in the receptive field of *every* later query, the maximal, content-free exposure. We hypothesize that this

	Type-1	Type-2 (no-op)		broadcast
scheme	(pos-0)	punct.	low-surp.	entropy
base	0.368	0.146	0.148	4.51
triangle-CPT	0.383	0.137	0.138	4.43
windowed	0.151	0.323	0.354	5.18
startup	**0.004**	**0.396**	**0.449**	**5.52**

Table 2: Where attention goes (Qwen2.5-0.5B; all schemes measured identically at a *genuine* 256-token window over 512-token sequences). Fraction of attention received by the position-0 token (Type-1) vs. low-information no-op tokens (Type-2), the latter measured two ways, a fixed punctuation list and the bottom-20% unigram-surprisal tokens, which agree, so the no-op sink is not an artifact of the punctuation proxy. Continued full-causal (triangle) training keeps (slightly intensifies) the Type-1 sink; windowed and startup training drain it onto the distributed Type-2 tokens and raise the broadcasting entropy. The attention–surprisal correlation sharpens in step (-0.07 at base $\rightarrow$ -0.16 at startup): as Type-1 dies, attention concentrates more on low-information tokens.

imbalance, early queries starved of context, position 0 always available, biases training to recruit the earliest positions as a stable anchor, producing the Type 1 sink. Consistent with this, the measured sink *grows* with context (Fig. 4): the model sinks *against* the data-side first-token importance ($\propto 1/C$, which *shrinks* with context), the signature of an active, mask-induced mechanism rather than a property of the data.

Learned and removable. Measuring the position-1 sink before and after finetuning shows it is a learned behaviour. On Qwen2.5-0.5B, a majority of softmax heads ($\approx 57\%$) sink onto token 1 in the base and full-attention-finetuned models; windowed finetuning drives this to 0% (Table 4), and the **startup** variant relocates the sink onto the trainable prompt rather than the content. On the larger hybrid model the absolute sink is weaker but the *direction* is identical: full-attention finetuning *builds* the sink, windowed finetuning *shrinks* it (Fig. 5).

Windowed finetuning recovers full-context quality. Scoring held-out perplexity as a function of the deployed window shows the expected collapse for a base model windowed at inference, and a near-flat curve for the windowed-finetuned model (Fig. 6): a model trained for a 256-token window matches the base model's full-context quality while attending to $8\times$ fewer tokens (Table 5). Continued pretraining under *full* attention does not help, it amplifies sink reliance, whereas the windowed objective removes it.

Scaling to a hybrid model. Qwen3.5-9B is a hybrid: of its 32 layers, only 8 (every 4th) are softmax attention; the rest are linear/Mamba-style attention (Gu and Dao 2024; Yang, Kautz, and Hatamizadeh 2024). The sink, and the windowing question, therefore concern only those 8 layers, $3/4$ of the network is already a constant-memory mechanism. We full-finetune just the 8 softmax blocks (freezing the 24 linear layers; 8-bit AdamW with gradient checkpointing on a single

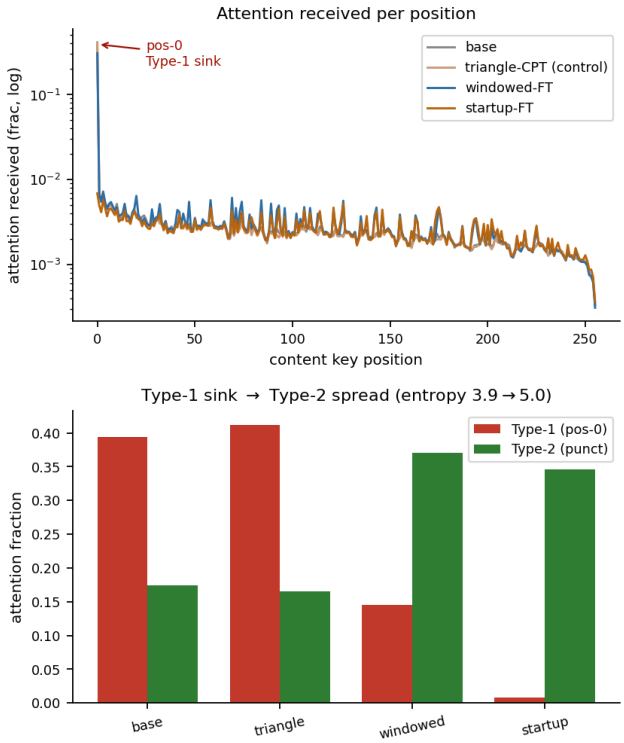

Figure 3: Where attention goes (Qwen2.5-0.5B, all schemes at a genuine 256-token window). **Top:** continued full-causal (triangle) training keeps the Type-1 pos-0 sink; windowed and startup training drain it and the mass moves onto distributed Type-2 no-op tokens, identically whether Type-2 is a punctuation list or the bottom-20% unigram-surprisal tokens (the two bars agree), so it is not a punctuation artifact. **Bottom:** broadcasting entropy rises and the attention–surprisal correlation sharpens ($-0.07 \rightarrow -0.16$) as the positional sink dies.

48 GB GPU). Even on this model, bounding the softmax window to 256 tokens collapses the base from a full-context perplexity of 6.5 to 20.0; windowed finetuning restores it to 6.3, full-context quality at a 256-token softmax window, and, unlike the baselines, gains nothing from StreamingLLM sink tokens because it no longer depends on them (Table 6).

Train small, deploy anywhere. Evaluating every windowed model at every inference window (Table 6) reveals an asymmetry. A model trained at a *small* window (windowed-128) is nearly flat across *all* deployment windows ($7.44 \rightarrow 6.62$): having learned to predict from 128 tokens, it is robust to any budget. A model trained at a *wide* window is fragile when deployed narrower (windowed-1024 rises to 24.7 at deploy-128), because it learned to lean on context that is no longer present. A model is always fine deployed *wider* than it was trained, and breaks deployed *narrower*: the practical rule is to train at the smallest window one might deploy.

	structural (share)		functional ($\|v\|$/mean)	
scheme	Type-1	Type-2	at T-1	at T-2
base	0.37	0.08	0.38	0.92
triangle-CPT	0.38	0.07	0.41	0.89
windowed	0.16	0.25	0.43	0.80
uniform-window	0.21	0.08	0.45	0.91
startup	0.004	0.32	0.88	0.81
persist	0.005	0.10	0.86	0.89

Table 3: Structural × functional transition across training schemes (Qwen2.5-0.5B, streaming eval sliding a $W{=}256$ window through the sequence). *Structural:* attention share on the concentrated first token (Type-1) vs. the sparse separators (Type-2). *Functional:* relative value norm at each sink ($\|v\|$/mean; low ⇒ no-op, after Fesser et al. 2026). The concentrated Type-1 sink carries a *low* value norm ($\sim$0.4, no-op-like); the sparse Type-2 separators a near-normal one ($\sim$0.8–0.9, value-carrying / broadcast). As windowed and startup training drain Type-1, attention moves onto the higher-value Type-2, concentrated→sparse and no-op→broadcast in lockstep. (Caveats: the pos-0 sink is heavily attended, so despite its low value it still injects $\sim$15% of base-model output energy, not a *pure* no-op; and uniform-window reduces Type-1 without the Type-2 spread, sinking on each window's relative boundary instead.)

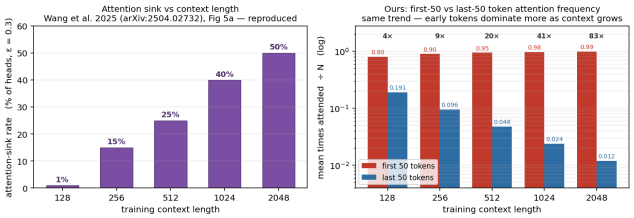

Figure 4: The measured Type 1 sink grows with context length, the opposite of the data-side first-token importance ($\propto 1/C$), the model sinks *against* the data, an artifact of the mask (H2).

A strict softmax-vs-gated control. Is the gain from *windowing*, or merely from more finetuning? We answer with a strict control: two 1.3B models from the flash-linear-attention suite trained on the *same* 100B tokens with the *same* skeleton, differing only in the token-mixer, a softmax Transformer and a gated-linear (GLA) model, evaluated across the full train-window × deploy-window grid (Table 7). The softmax base collapses at tight windows ($10.2 \rightarrow 17.9$ at deploy-128); continued *full-causal* pretraining (triangle-CPT) only partly recovers (14.0, the control held fixed); windowed finetuning gives near-flat rows, the same *train small, deploy anywhere* asymmetry as the hybrid model, so the recovery is from the window, not the extra training. The gated GLA has no attention window: in its native $O(1)$-state mode it reaches 8.71 with constant memory, beating every windowed-softmax cell, but only because it always uses full context. A pure recurrence has no honest hard-W window, the only way to impose one is to reset its state, which no real deploy-

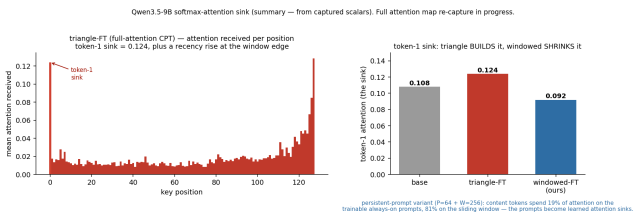

Figure 5: Qwen3.5-9B softmax-layer sink. Full-attention (triangle) finetuning builds the token-1 sink to 0.124; windowed finetuning shrinks it to 0.092; base is between (0.108). The persistent-prompt variant relocates ∼19% of every content token's attention onto trainable always-on registers.

	full attention (learned tendency)	windowed mask (as deployed)
base + history	57.6%	0.0%
sliding-history (trained)	**5.8%**	0.0%

Table 4: Position-1 sink on Qwen2.5-0.5B given the *same* real history (63 history + 256 content, W=64): % of softmax heads with mean attention > 0.3 to the history's first token. Windowed finetuning cuts the learned sink ∼10× (57.6%→5.8%); the window masks it to 0% at deployment.

ment does, so the legitimate comparison is bounded-$O(W)$ sliding-window softmax against native-$O(1)$ gating. There the windowed-softmax retrofit reaches the gated model's quality at a moderate budget, while the gated model needs no windowing at all.

Compute. The payoff is efficiency. Full softmax attention is $O(L^2)$; the windowed and gated mechanisms are $O(L)$. At a 32k context the windowed model runs 2.7× faster than full attention (Table 8), with linear rather than quadratic growth.

3.3 H3: Removing Type-1, the Model Resorts to the Separators

Hypothesis. If the Type 1 anchor is removed, e.g. by a windowed objective that takes position 0 out of every deep query's receptive field, the storage/no-op function is not lost. We hypothesize that, rather than anchoring the shared representation on the beginning of the sequence, later tokens *resort to* the Type 2 low-information separators, repurposing them to carry the role position 0 used to play and redistributing attention onto them.

Removing the concentrated sink does not destroy the attention mass, it *relocates* it. The same measurement (Table 2), read causally, shows that as Type-1 drains across triangle→windowed→startup (0.38 → 0.15 → 0.004) the sparse Type-2 mass *rises* in lockstep (0.14 → 0.32 → 0.40), with the broadcasting entropy climbing 4.4 → 5.5: with the positional anchor suppressed, the model resorts to the content-based no-op separators as its store. We note the boundary, addressed under H4: this broader spread is a *distributional* property, not long-range retrieval.

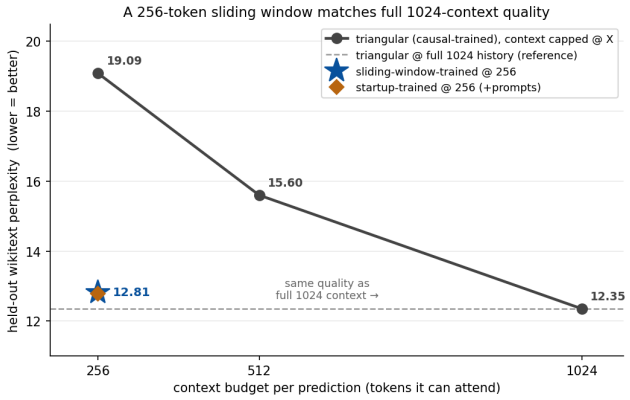

Figure 6: Held-out perplexity vs. deployed working-memory window (Qwen2.5-0.5B). The base model collapses as the window shrinks; the windowed-finetuned model is nearly flat and matches full-context quality at a small window.

model (Qwen2.5-0.5B)	context budget	held-out ppl
triangular (causal-trained)	256	19.09
triangular (causal-trained)	512	15.60
triangular (causal-trained)	1024 (full)	**12.35**
sliding-window-trained	256	**12.81**
startup-trained	256 (+ prompts)	12.78

Table 5: Held-out wikitext perplexity vs. context budget on Qwen2.5-0.5B. The causal-trained model needs the full 1024-token history (12.35) and degrades steeply when windowed (19.09 at 256, worse than the untrained base, because windowing masks the first-token sink it leans on); the windowed-finetuned model reaches 12.81 at a 256-token window, matching the causal model's full-context quality on a quarter of the context.

3.4 H4: Separator Sinks Support Streaming

Hypothesis. Because Type 2 tokens are distributed through the text, each one sitting at the end of a span of already-accumulated context, we hypothesize that attention parked on them *absorbs* that preceding-token information, building a *hierarchical* representation (separators at clause-, sentence-, and paragraph-scope summarizing progressively larger spans) with more granular control over information broadcasting than a single distant position-0 anchor. In a bounded-window deployment a recent separator is always inside the window and has already integrated its local context, making it a better relay point than a position-0 sink the window has scrolled past, so a constant-working-memory model should not merely tolerate the loss of the Type 1 sink but *benefit* from leaning on Type 2 relays, improving streaming performance.

Persistent prompts as trainable sinks. The persistent-prompt variant (P=64 always-on prompts + a W=256 window) lets content tokens spend a controllable fraction of attention on the learned registers: in our run, 19% of every content token's attention goes to the 64 persistent prompts and 81% to the window. The prompts thus become explicit,

	perplexity at inference window				
model	128	256	512	1024	2048
untrained baseline					
base 9B	59.8	28.1	13.8	7.80	7.26
+ StreamingLLM	9.33	8.47	7.91	7.58	7.28
continued-pretraining baseline					
triangle-FT	33.7	18.2	8.66	6.96	6.59
+ StreamingLLM	7.88	7.39	7.03	6.83	6.56
ours, windowed (one model per training window)					
windowed-128	**7.44**	7.14	6.94	6.83	6.62
windowed-256	7.82	**7.19**	6.93	6.79	6.57
windowed-512	11.5	7.82	**6.95**	6.79	6.55
windowed-1024	24.7	12.7	7.44	**6.83**	6.54
windowed-2048	27.6	14.5	7.76	6.84	**6.53**
ours, startup (windowed + cold-start prompt)					
startup-256	7.82	7.17	6.92	6.79	6.57

Table 6: Qwen3.5-9B held-out perplexity, train-window (rows) $\times$ inference-window (columns); bold is native (train $=$ deploy). Windowed models beat both baselines and their StreamingLLM versions at tight windows, with no sink tokens. Small-window models are robust to any deployment window; wide-window models break when deployed narrower (*train small, deploy anywhere*).

trainable attention sinks, the function the first token was being abused for, now a designed component. On perplexity the persistent and startup variants track the plain windowed model, with a small edge when deployed *narrower* than trained (the prompt fills the coldest part of the window). The registers matter most on a downstream task that needs distant context, which we test with a *fair* control: triangle (full-mask), windowed, and persist models all continued-pretrained on the same wikitext for the same steps, so only the mask differs (Table 9). At a 256-token window the full-mask control collapses (HotpotQA generation F1 0.036), windowed training recovers it (0.095), and the persistent-prompt model reaches 0.166, matching the base+StreamingLLM post-hoc trick (0.166), but as a trained constant-memory model. This marks the boundary of the broadcasting effect: windowed training spreads attention onto the Type-2 tokens (a distributional change) but does not by itself relay a *specific* distant fact; the persistent registers are what restore long-range anchoring, unifying constant-memory streaming with retrieval in one trained model.

Downstream long-context QA. To isolate the mask effect from task learning we also continued-pretrain (no QA supervision) and evaluate zero-shot on LongBench multi-hop QA (Bai et al. 2024). At a tight 256-token window, the windowed model uses its bounded memory markedly better than a full-attention-pretrained control, and StreamingLLM helps the baselines but not the windowed model, the same signature as in perplexity, on a real task (Fig. 7).

4 Related Work

Attention sinks and streaming. Xiao et al. (2024) identified the sink and exploited it for streaming inference by retaining initial tokens; Han et al. (2024) use a similar fixed pattern for length generalization. These keep the sink at inference; we remove the dependency by finetuning, and (in the persistent-prompt variant) replace the accidental sink with trainable registers.

	perplexity at inference budget				
model	128	256	512	1024	full
softmax, untrained / control					
base	17.9	14.9	12.8	11.3	10.2
triangle-CPT	14.0	12.1	10.8	9.61	8.80
softmax, ours: sliding-window (one model / train window)					
windowed-128	**10.6**	9.70	9.30	9.10	9.06
windowed-256	10.8	**9.63**	9.17	8.96	8.92
windowed-512	11.9	9.85	**9.15**	8.89	8.84
windowed-1024	12.7	10.5	9.36	**8.88**	8.81
gated (GLA), same data & skeleton					
gla-CPT	8.71 (window-invariant, $O(1)$ memory)				

Table 7: Strict control on the flash-linear-attention 1.3B pair (same GPT skeleton, same 100B tokens; only the token-mixer differs, softmax vs. gated-linear), held-out perplexity, training budget (rows) $\times$ inference budget (columns); bold marks each softmax model's native window. **Softmax:** the base collapses at tight windows; the full-mask (triangle) control only partly recovers; sliding-window finetuning gives near-flat rows, *train small, deploy anywhere*. **Gated:** GLA continued-pretrained on the same data (gla-CPT) is *window-invariant* (8.71 at $O(1)$ memory): it has no attention window to tighten, so it runs at a single operating point. The honest comparison is therefore bounded-$O(W)$ sliding-window softmax vs. native-$O(1)$ gating, the windowed-softmax retrofit reaches the gated model's quality at a moderate budget, while the gated model needs no windowing at all. (A pure recurrence cannot be given a hard W-window without resetting its state, which no real deployment does; we therefore do not report a "windowed gla".)

Windowed and efficient attention. Sliding-window attention is used architecturally by Longformer (Beltagy, Peters, and Cohan 2020) and Mistral (Jiang et al. 2023), and linear/state-space models (Gu and Dao 2024; Yang, Kautz, and Hatamizadeh 2024) achieve constant memory by construction; Qwen3.5 is a hybrid of the two. We instead *finetune* a pretrained softmax model into a windowed one and study what the sink does under that change. Position-extrapolation methods such as ALiBi (Press, Smith, and Lewis 2022) share the "train short, test long" spirit; we observe a complementary "train small, deploy anywhere" rule for the attention window.

Context as weight. Our framing, that windowing forces older context into the parameters, connects to context distillation and knowledge editing; here the mechanism is attention geometry rather than an explicit distillation loss.

seq. length	full	windowed	gated
8k	313	226	258
16k	797	436	499
32k	2363	**866**	989

Table 8: Forward latency (ms) vs. sequence length (fla 1.3B, one A6000). Full softmax attention scales quadratically; windowed and gated scale linearly, at 32k the windowed model is 2.7× faster than full.

variant (CPT)	deploy	F1	ans-ppl
triangle (full-mask)	windowed	0.036	112
windowed	windowed	**0.095**	32
persist	persist	**0.166**	21
base (no CPT)	full	0.305	2.8
base + StreamingLLM	streaming	0.166	13.6

Table 9: HotpotQA, *fair* mask-only control (Qwen2.5-0.5B). The three CPT variants see the *same* wikitext for the *same* steps, only the attention mask differs, so QA-ability drift is held constant. At a 256-token window, the full-mask (triangle) control collapses (F1 0.036); windowed training recovers it (0.095); the persistent-prompt variant reaches 0.166, *matching the base+StreamingLLM post-hoc trick* as a trained constant-memory model. SQuAD-normalised generation F1.

5 Conclusion

The attention sink is a learned consequence of the triangular causal mask, and it is exactly what makes a pretrained model fragile to a bounded context. Finetuning under a sliding-window mask removes the dependency: the model acquires a constant working memory, matches full-context quality at a small window, and, unlike inference-time fixes, needs no sink tokens. Two design points fall out: "train small, deploy anywhere," and the persistent-prompt variant that turns the accidental sink into a controllable set of trainable registers. We view this as evidence that, for the long-range part of a model's behaviour, context can be traded for weight.

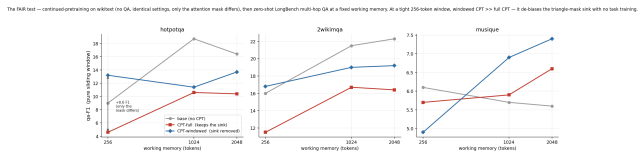

Figure 7: LongBench multi-hop QA (Qwen2.5-0.5B, continued-pretraining, no task supervision). At a tight window the windowed model uses a bounded working memory better than a full-attention control; the gap shrinks as the window grows.

References

Bai, Y.; Lv, X.; Zhang, J.; et al. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

Beltagy, I.; Peters, M. E.; and Cohan, A. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150*.

Fesser, L.; Jacobs, M.; Fel, T.; Keller, A.; and Kakade, S. 2026. A Unifying View of Attention Sinks: Two Algorithms, Two Solutions. *arXiv preprint arXiv:2606.08105*.

Gu, A.; and Dao, T. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. In *Conference on Language Modeling (COLM)*.

Han, C.; Wang, Q.; Peng, H.; et al. 2024. LM-Infinite: Zero-Shot Extreme Length Generalization for Large Language Models. In *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Merity, S.; Xiong, C.; Bradbury, J.; and Socher, R. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations (ICLR)*.

Press, O.; Smith, N. A.; and Lewis, M. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations (ICLR)*.

Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.

Yang, S.; Kautz, J.; and Hatamizadeh, A. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.